

COMS30030 - Image Processing and Computer Vision



Lecture 06

Object Detection

Majid Mirmehdi | majid@cs.bris.ac.uk

What is 'Object Detection'?

- Object detection aims at bridging the 'semantic gap' between...
 - given pixel values, *and*
 - meaningful objects (grouping of pixels + classification of groups)

Image regions need to be found and assigned with **semantic labels** from a space of object classes



What is 'Object Detection'?

Why do classical shape detection and segmentation on their own rarely work for real-world object detection?

- high intra-class variance
- low inter-class variance
- classes are rarely well defined

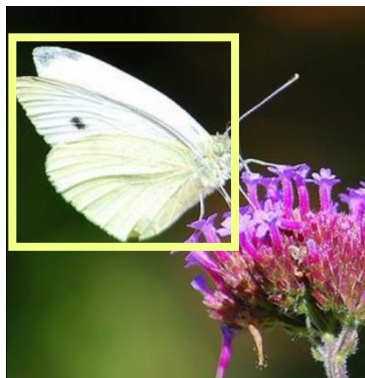
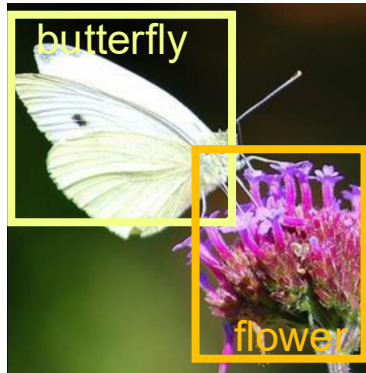


- change of illumination, scale, pose, deformation, occlusion...

Terminology

Classification → butterfly

Multiple
object
detection



object detection =
Classification + localisation



Semantic Segmentation



Panoptic Segmentation

Object Detection Techniques

- **Line and circle detection:** Techniques like the Hough Transform can be used to detect lines and circles in an image, which can indirectly help locate objects with specific geometric shapes.
- **Colour-based detection:** In some cases, objects can be detected based on their colour properties. This is especially useful when objects have distinct and consistent colors.
- **Template matching:** Sliding a template over the input image and finding regions where the template best matches the local image content.
- **Classifiers with sliding window detectors:** Applying image classification on overlapped patches in the image.
- **Deep learning-based object detectors:** Object detector automatically learns image features required for detection tasks, and instance segmentation.

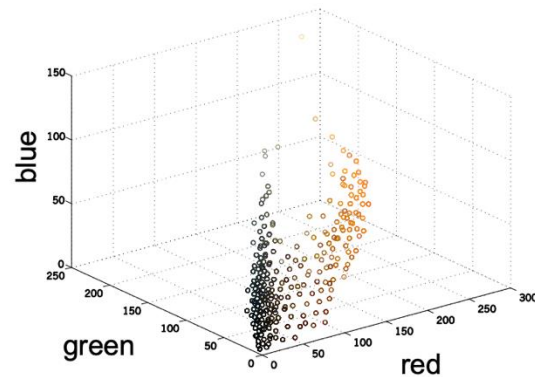
(out of scope in this unit)

Colour-based Detection

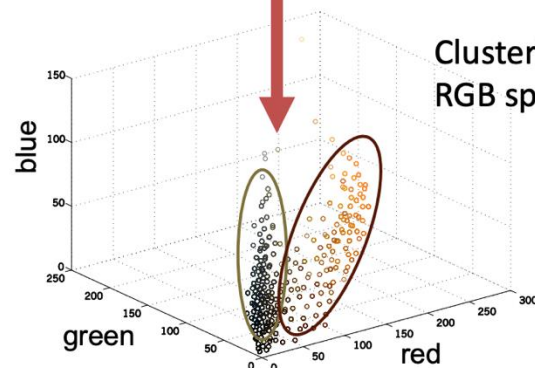


map to 3D

RGB space

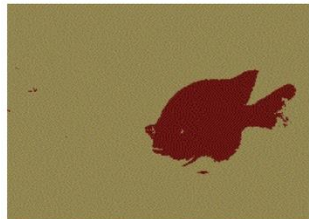


Clustering in
RGB space

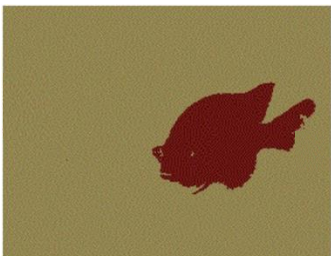


map back to

pixel space



Morphological
Operations



Morphological operations

What are they used for?

- Binary images (although version for greylevel images also exists)
- Can be used for **post-processing** segmentation results, e.g. noise filtering, enhancing object structure, ...
- Quantitative description of objects (area, perimeter, etc.)

Core techniques

Erosion

Dilation

Opening

Closing

Morphological operations

Two *sets*:

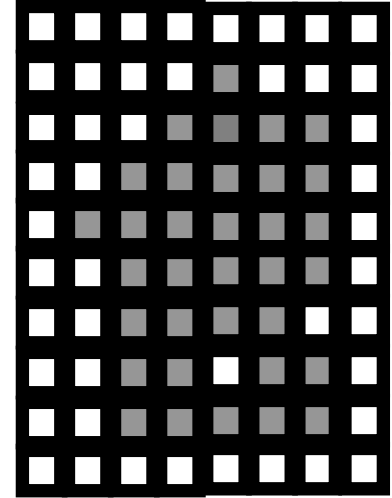
- Image
- Morphological **kernel** (*or structuring element*)

▪ Dilation (D)

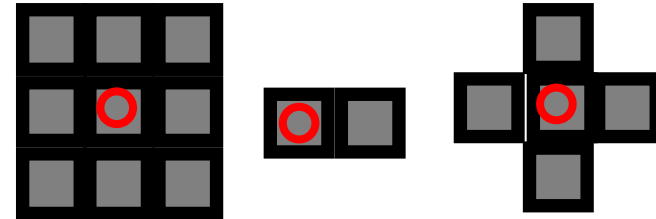
- Union of the **kernel** with the **image** set.
- Increases resulting area.

▪ Erosion (E)

- Intersection of the **kernel** with the **image** set.
- Decreases resulting area.



Example **kernels**



Dilation

Morphological dilation ' $\oplus$ ' combines two sets using vector of set elements

$$X \oplus B = \{p \in Z^2 \mid p = x + b, x \in X, b \in B\}$$

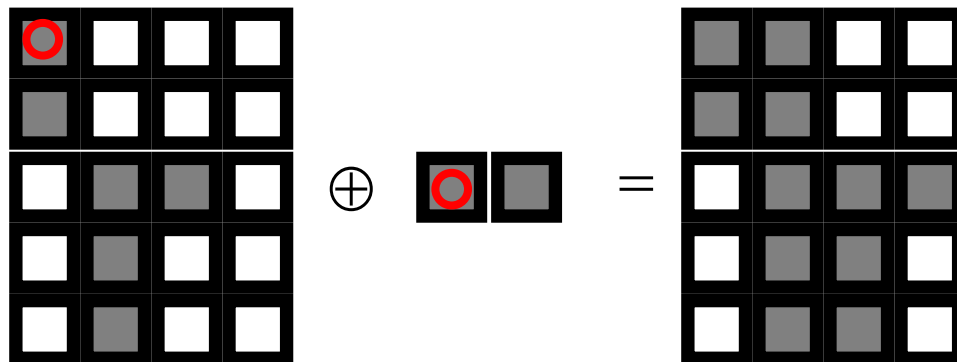
Commutative: $X \oplus B = B \oplus X$

Associative: $X \oplus (B \oplus D) = (X \oplus B) \oplus D$

Invariant of translation: $X_h \oplus B = (X \oplus B)_h$

Is an increasing transformation: If $X \subseteq Y$ then $X \oplus B \subseteq Y \oplus B$

The dilation $X \oplus B$ is the point set of all possible vector additions of pairs of elements, one from each of the sets X and B



Erosion

Morphological erosion ' $\ominus$ ' combines two sets using vector subtraction of set elements and is a dual operator of dilation

$$X \ominus B = \{p \in Z^2 \mid \forall b \in B, p + b \in X\}$$

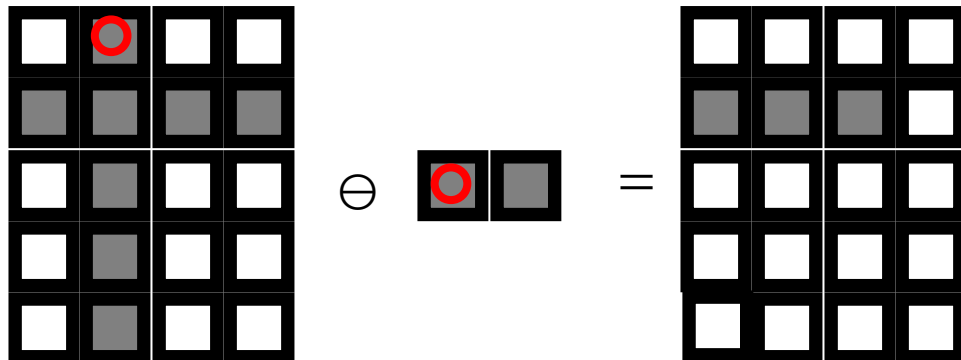
Not Commutative: $X \ominus B \neq B \ominus X$

Not associative: $X \ominus (B \ominus D) \neq (X \ominus B) \ominus D$

Invariant to translation: $X_h \ominus B = (X \ominus B)_h$ and $X \ominus B_h = (X \ominus B)_{-h}$

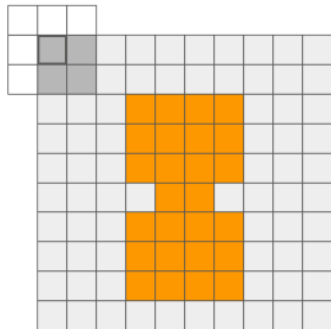
Is an increasing transformation: If $X \subseteq Y$ then $X \ominus B \subseteq Y \ominus B$

Every point p from the image is tested; the result of the erosion is given by those points p for which all possible $p + b$ are in X .

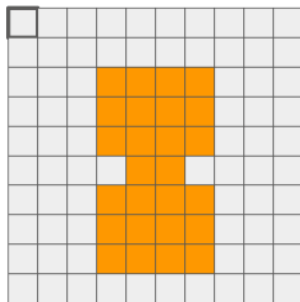


Dilation and Erosion examples

Dilation

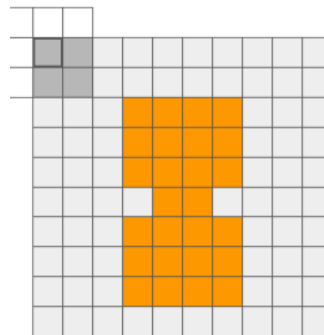


A=binary image

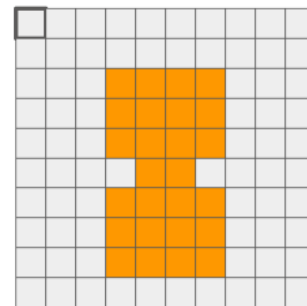


$B =$

1	1	1
1	1	1
1	1	1



A=binary image



Erosion

Examples

Original image



Eroded image

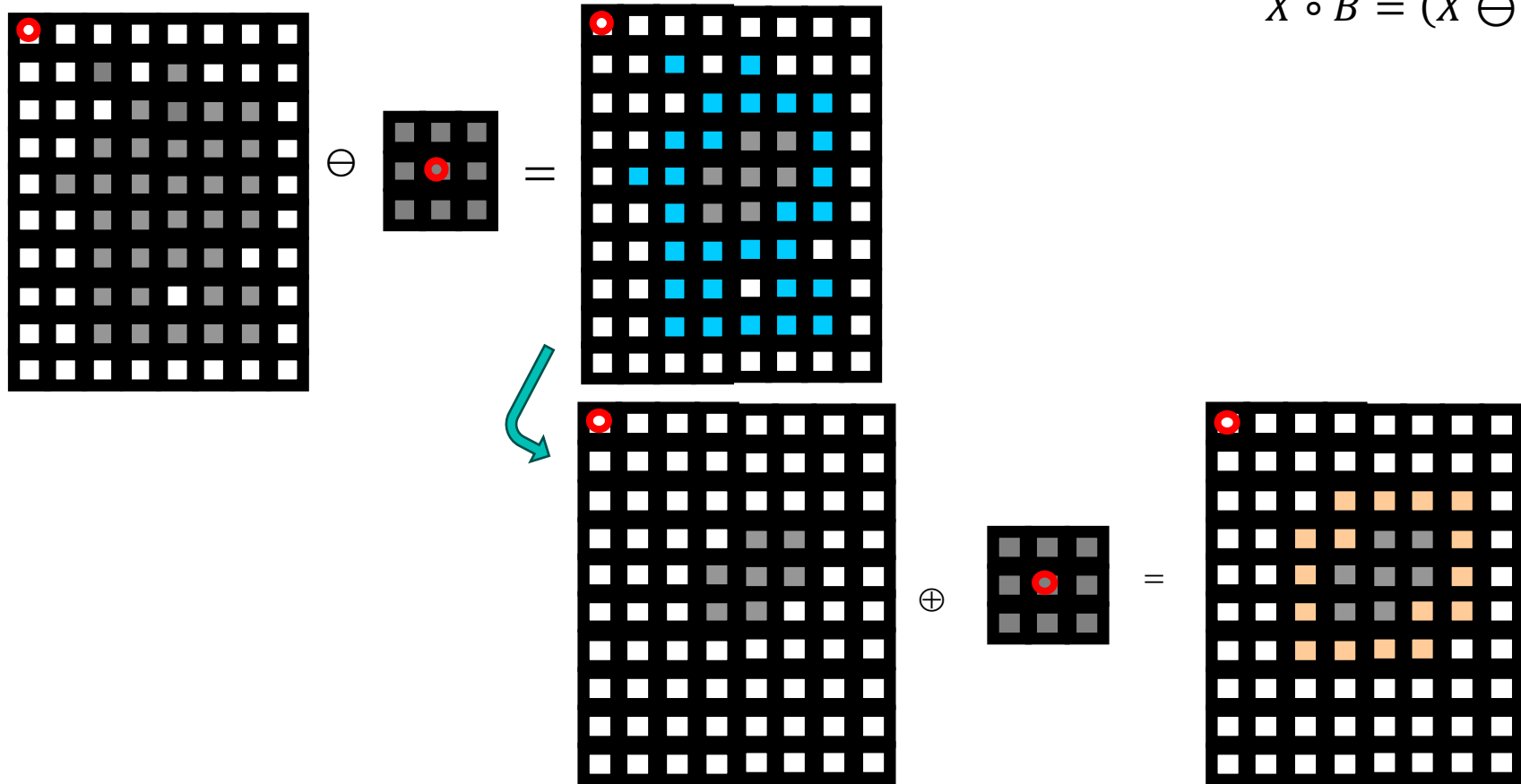


Dilated image



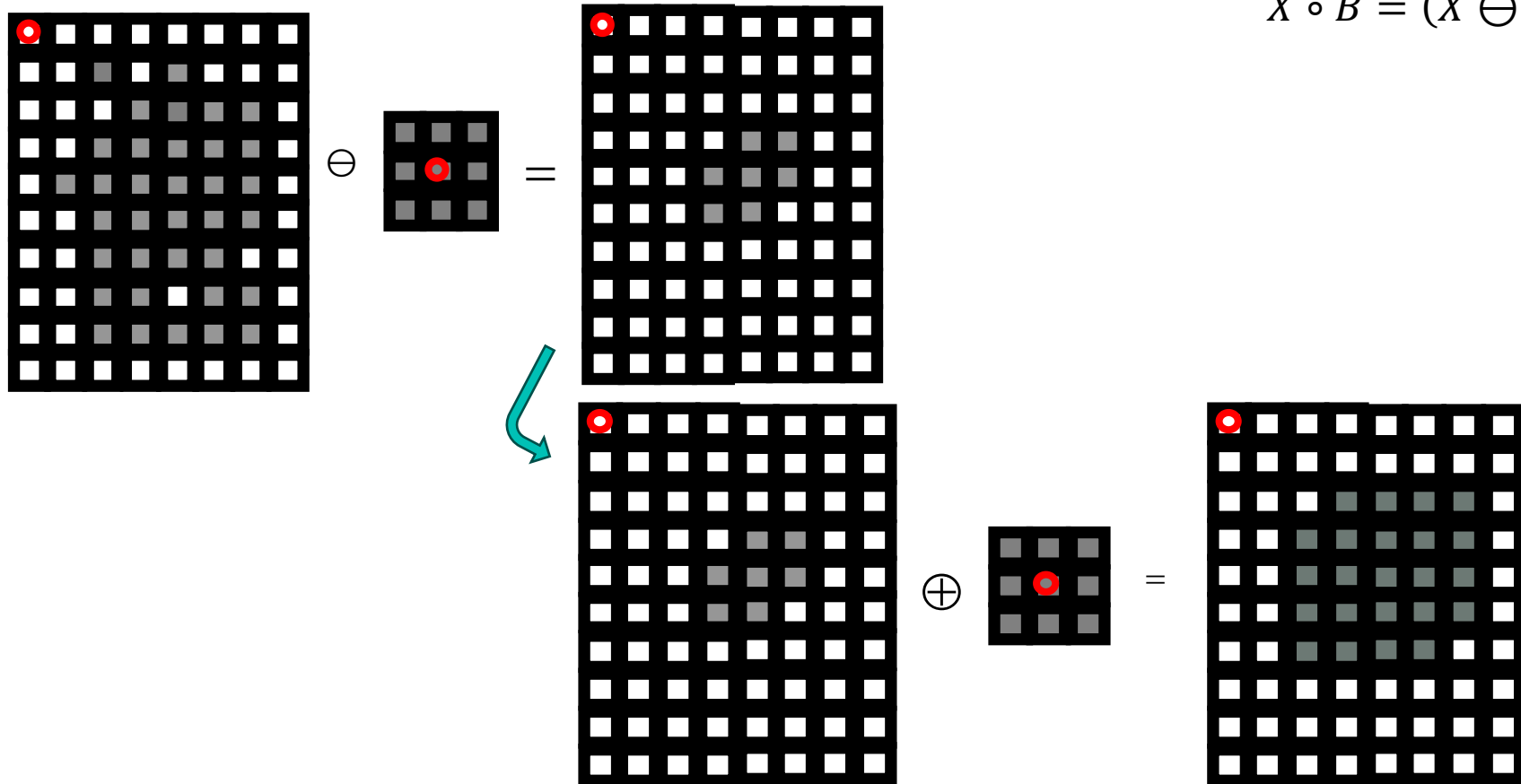
Opening: Erosion followed by Dilation

$$X \circ B = (X \ominus B) \oplus B$$



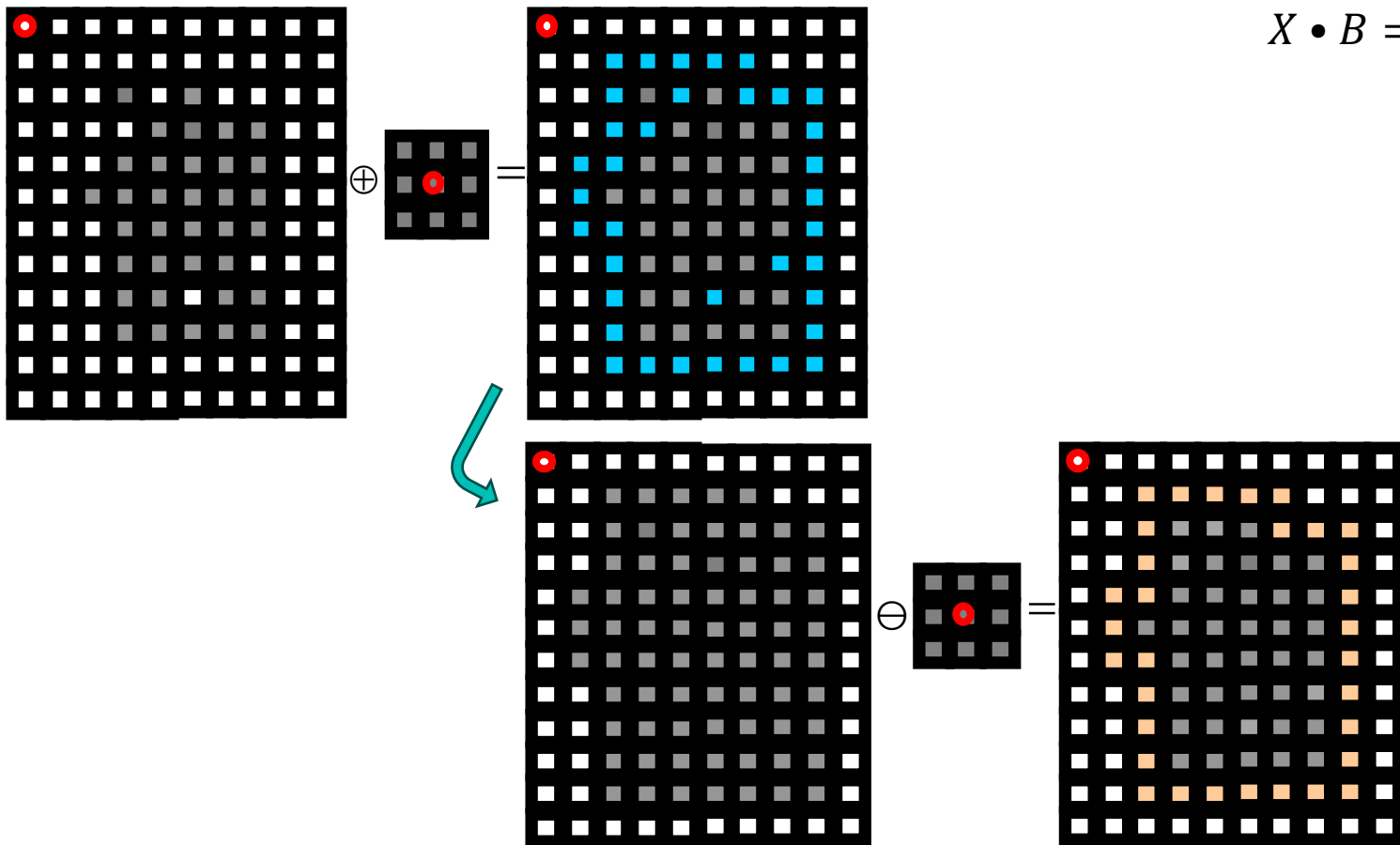
Opening: Erosion followed by Dilation

$$X \circ B = (X \ominus B) \oplus B$$

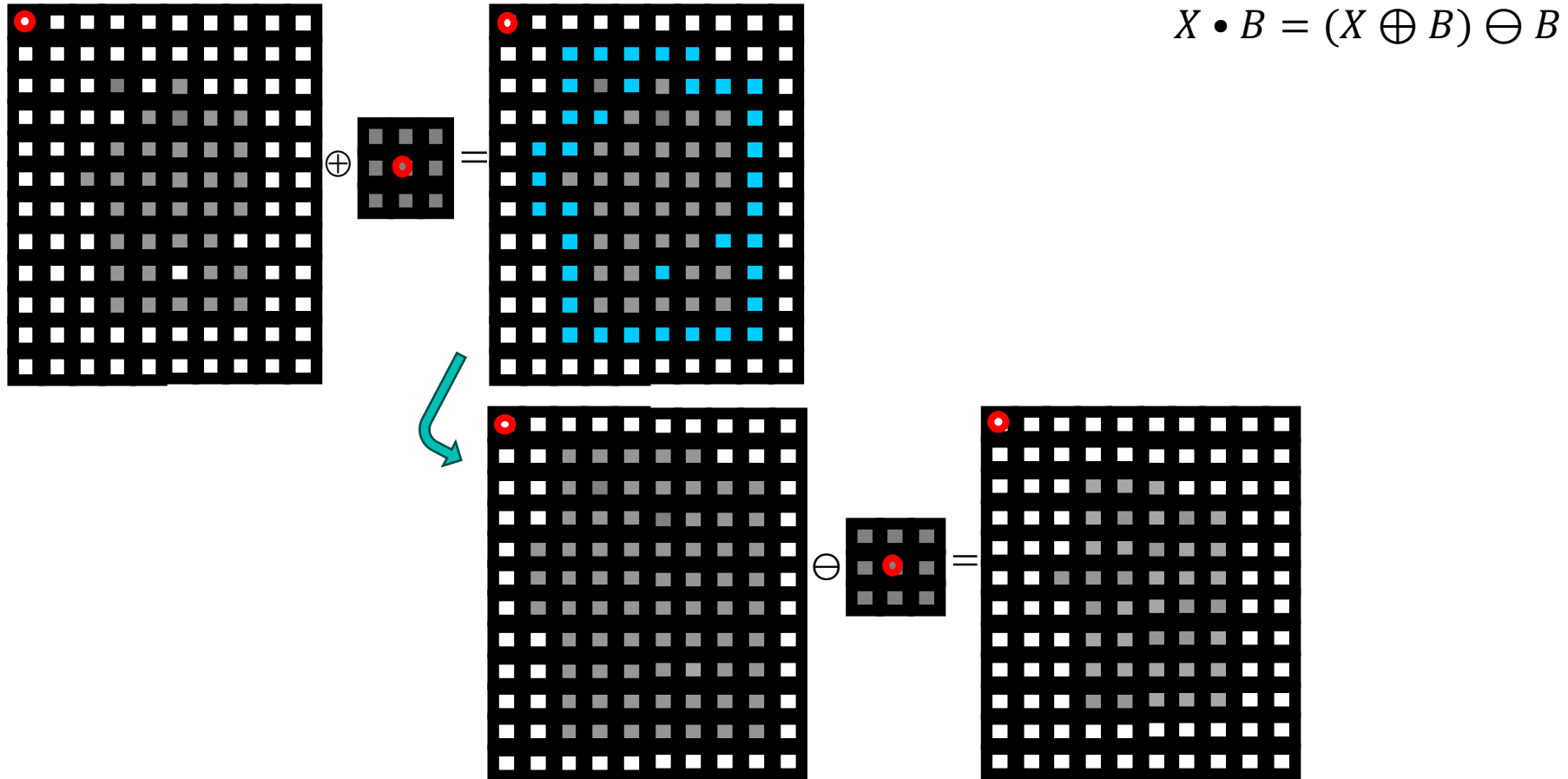


Closing: Dilation followed by Erosion

$$X \bullet B = (X \oplus B) \ominus B$$



Closing: Dilation followed by Erosion



Examples



image

erosion

dilation



image

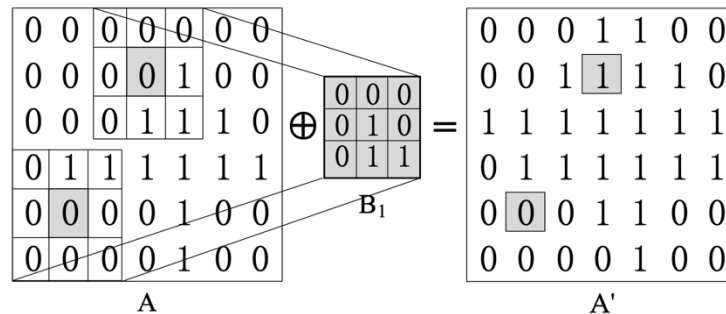
opening

erosion then
dilation

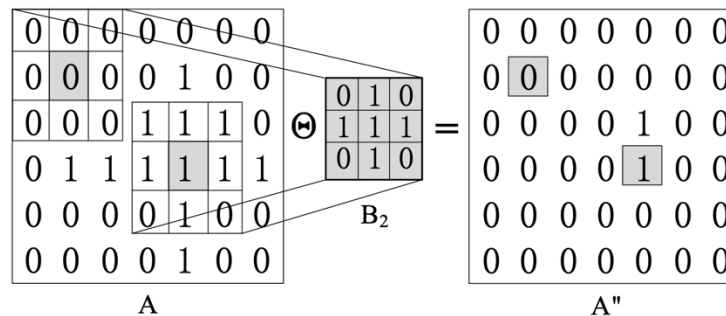
image

closing

dilation then
erosion

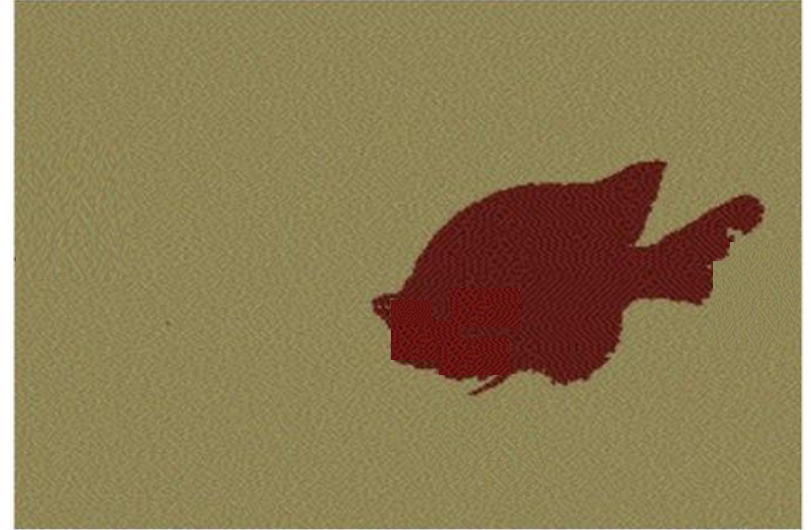
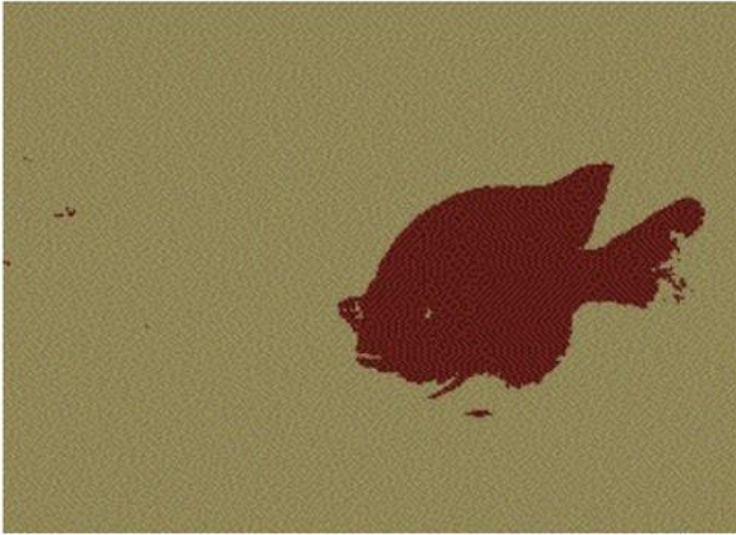


(a) Dilation operator $A \oplus B_1 = A'$



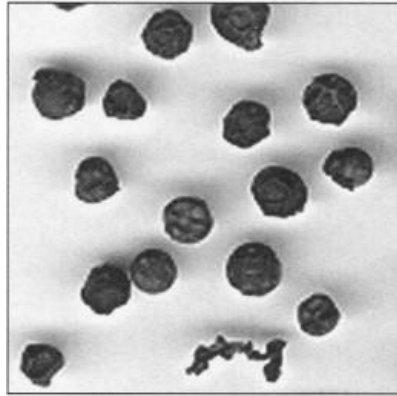
(b) Erosion operator $A \ominus B_2 = A''$

Example of Opening



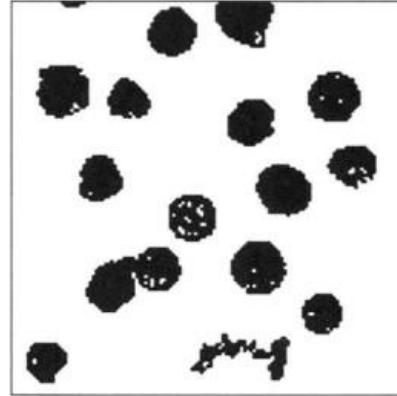
Example of Closing

(a) Image of peppercorns



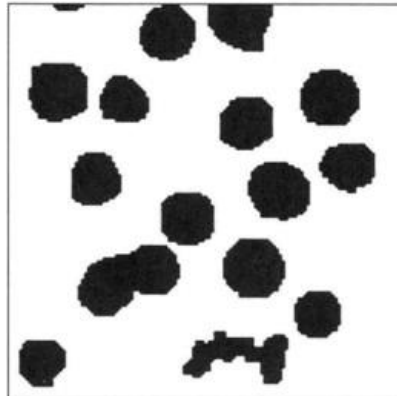
(a)

(b) Thresholded



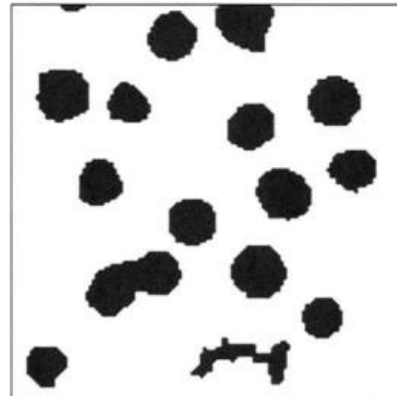
(b)

(c) 3x3 dilation...



(c)

(d) ...then 3x3 erosion



(d)

Example of Edge Detection!



Erosion as isotopic shrink.



Contours obtained by subtraction of an eroded image from the original.

Sliding Window Detectors

- Image is tested for object presence window-by-window
- The window is `slided` and `scaled` throughout the image



- Each resulting window is judged w.r.t. an object model giving a response indicating object presence or absence

Template Matching

- Find the best **similarity** (or the lowest **difference**) or within the defined threshold



pixel i in box y in the image,
 y has the same size as $\hat{y}$

pixel i in template $\hat{y}$

- Maximum

correlation:
$$\frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \mu_y}{\sigma_y} \right) \left(\frac{\hat{y}_i - \mu_{\hat{y}}}{\sigma_{\hat{y}}} \right)$$

Annotations:
 - y_i : pixel i in box y in the image
 - $\hat{y}_i$: pixel i in template $\hat{y}$
 - μ_y : mean
 - $\mu_{\hat{y}}$: mean
 - σ_y : std
 - $\sigma_{\hat{y}}$: std

- Minimum

mean absolute error:
$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

mean square error:
$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

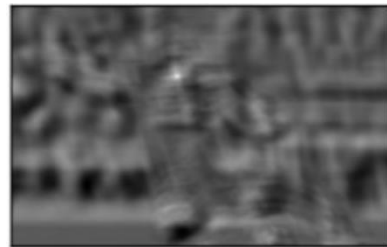
total number of pixels in the template box

Template Matching

- Find the best **similarity** (or the lowest **difference**) or within the defined threshold



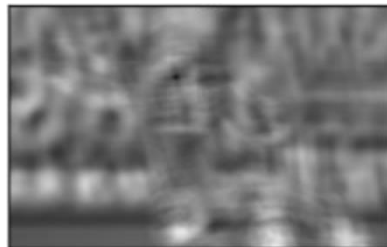
- correlation: $\frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \mu_y}{\sigma_y} \right) \left(\frac{\hat{y}_i - \mu_{\hat{y}}}{\sigma_{\hat{y}}} \right)$



Similarity map



- mean absolute error: $\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- mean square error: $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$



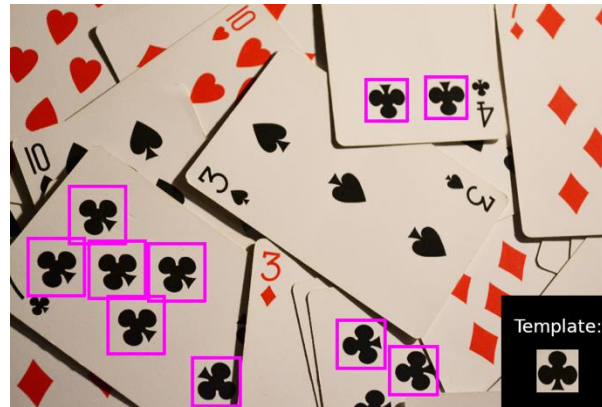
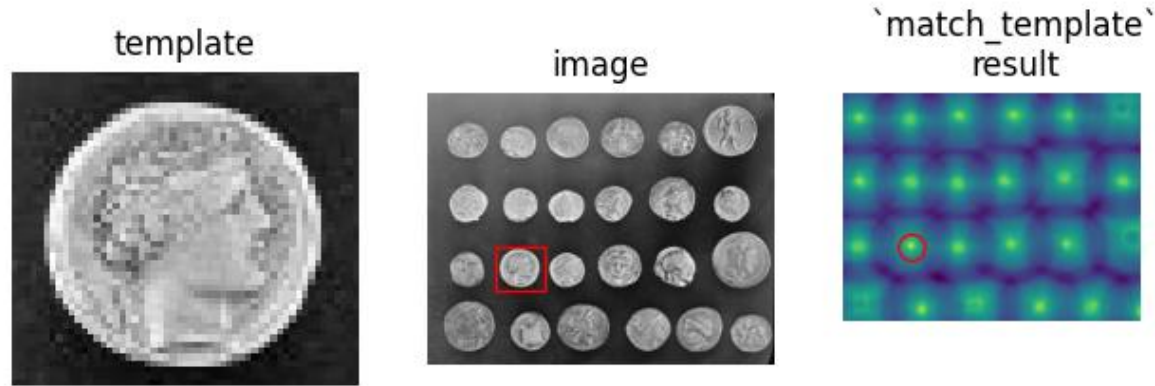
error map



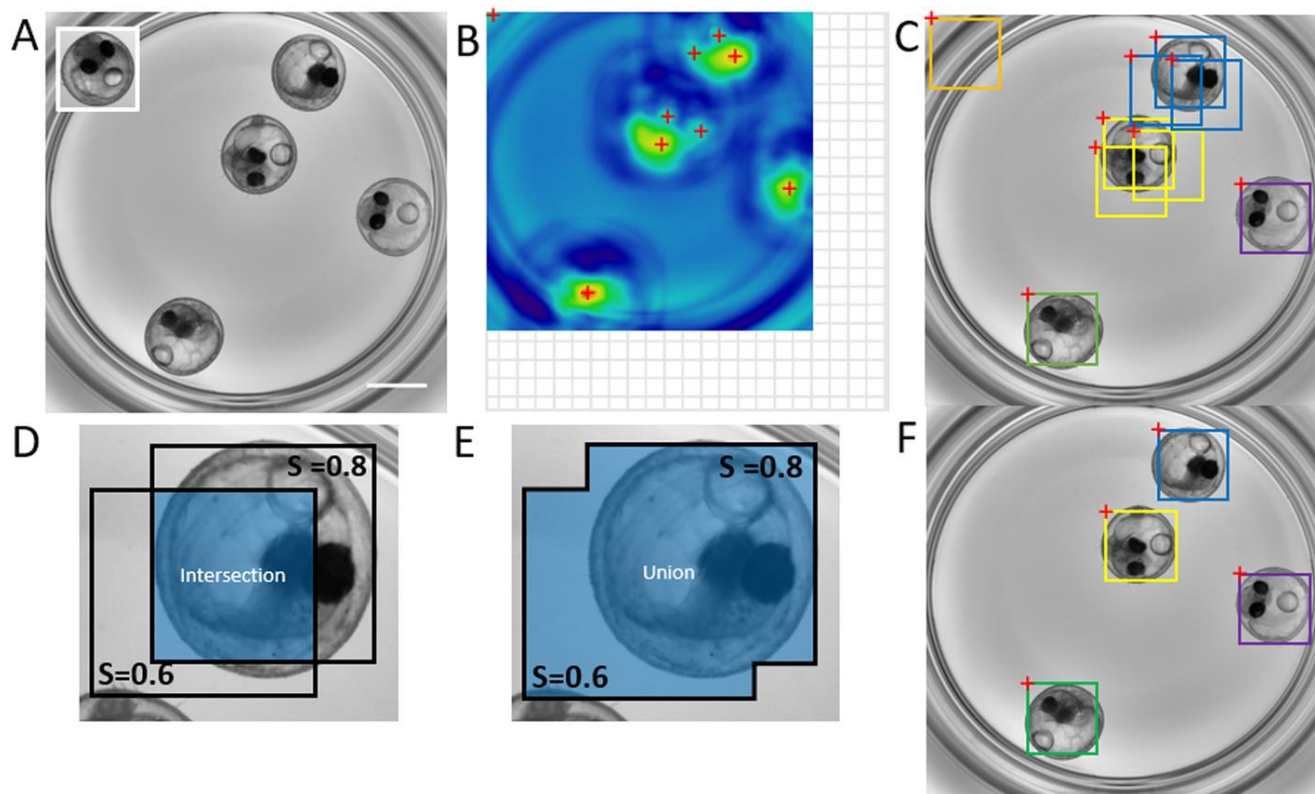
Template Matching can be expensive...

- Template image size: 53 x 48
- Source image size: 177 x 236
- Assumption: template image is inside the source image.
- Correlation (search) matrix size: 124 x 188
- Computation count: $124 \times 188 \times 53 \times 48 = 59,305,728$

Template Matching examples



Template Matching examples

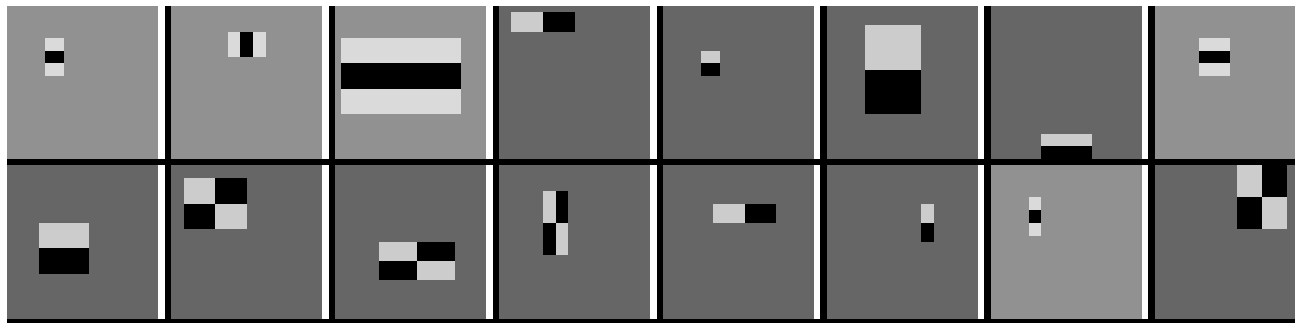


$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

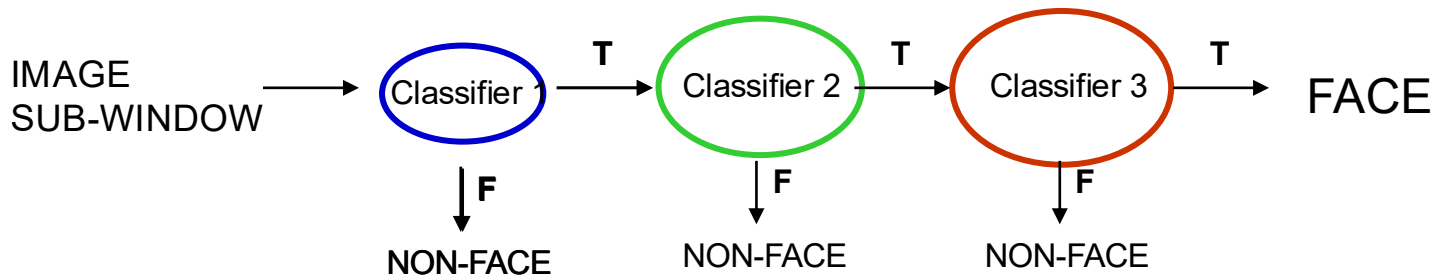
Same object (IoU closer to 1) or distinct objects that are close to each other (IoU closer to 0).

Viola-Jones: Another Sliding Window Approach

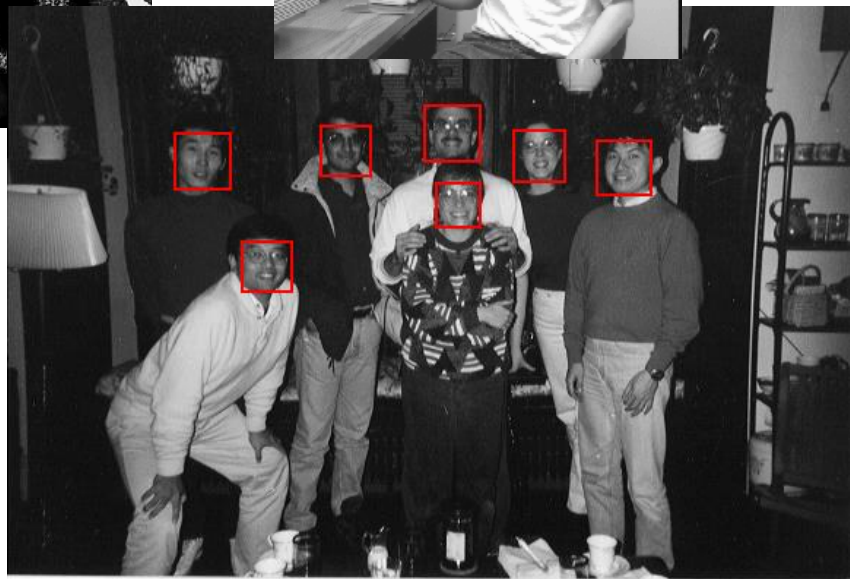
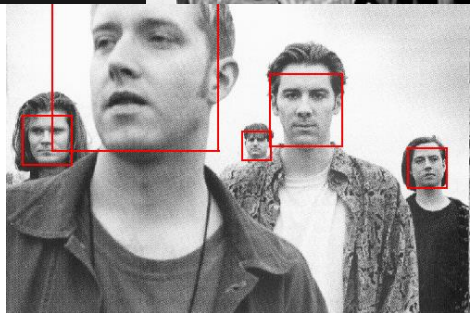
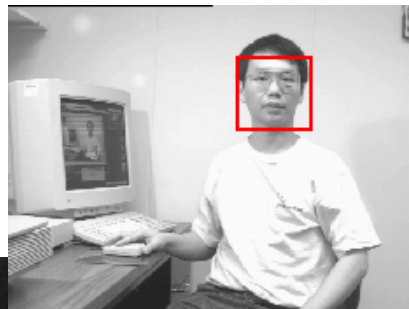
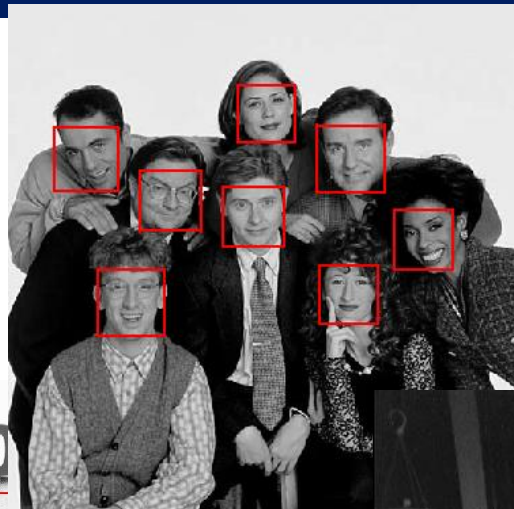
Hand-crafted weak features, but computationally efficient, calculated in sliding windows...



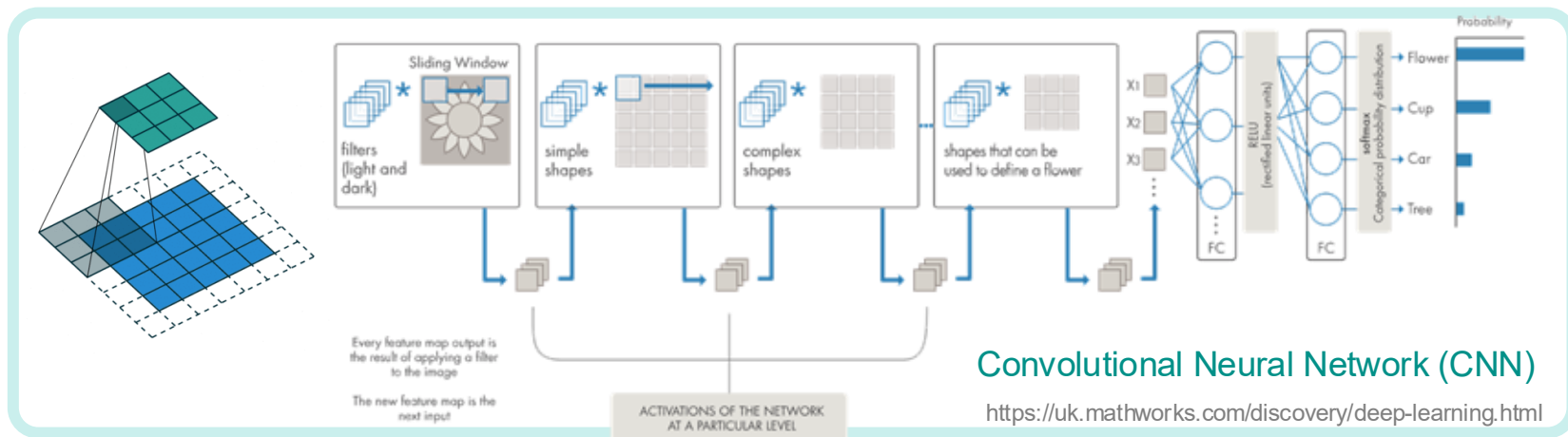
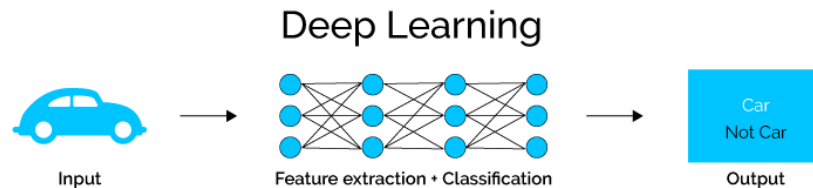
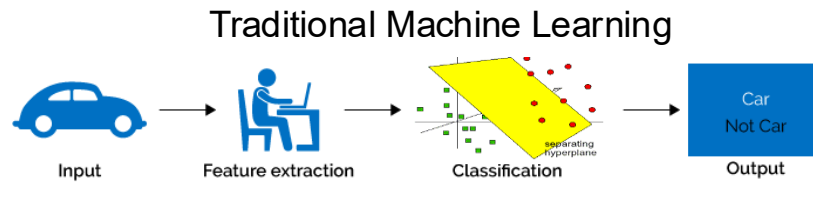
Construct a cascade of classifiers, which can reject most of the negative examples at early stages of processing, thereby significantly reducing computation time.



Viola-Jones: Another Sliding Window Approach



Next year: Deep learning-based object detectors

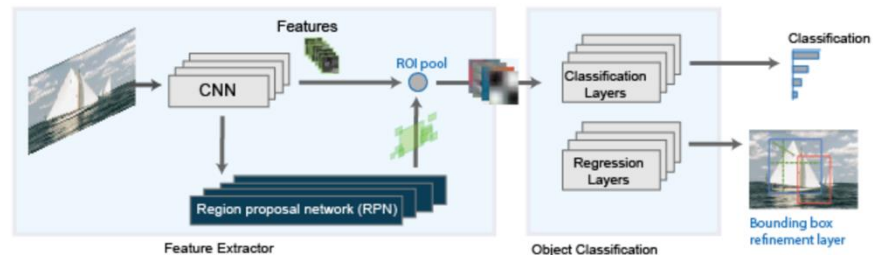


Next year: Deep learning-based object detectors

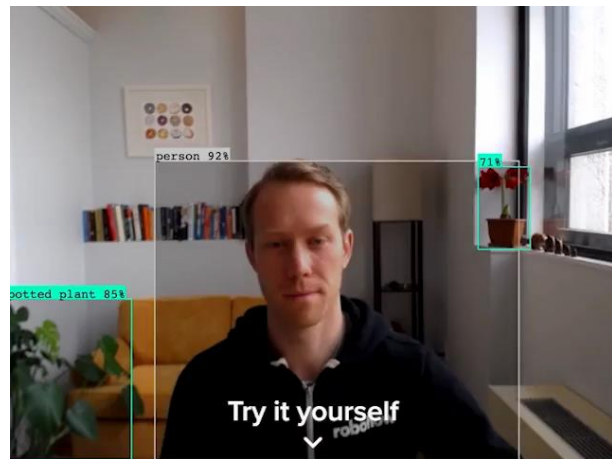
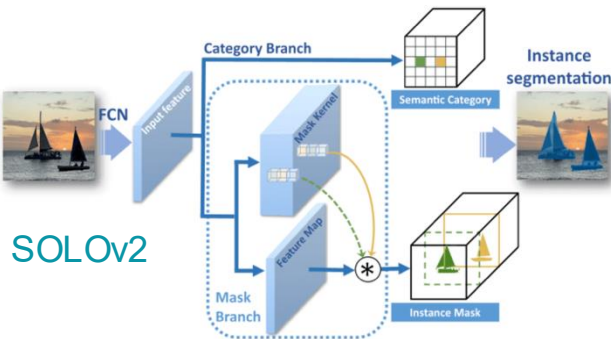
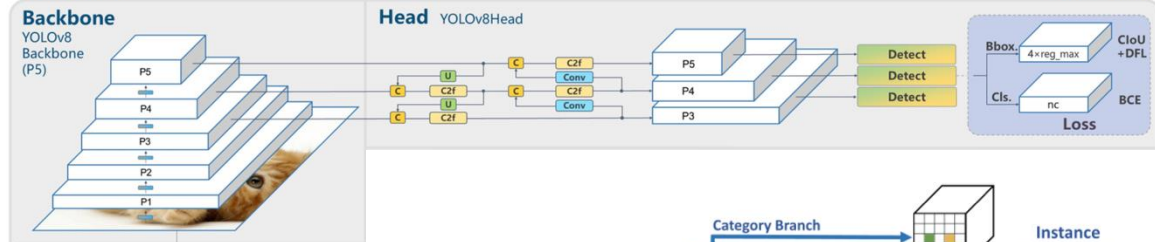
Objective



Faster-RCNN



YOLOv8, YOLOv5, YOLOv6, YOLOX



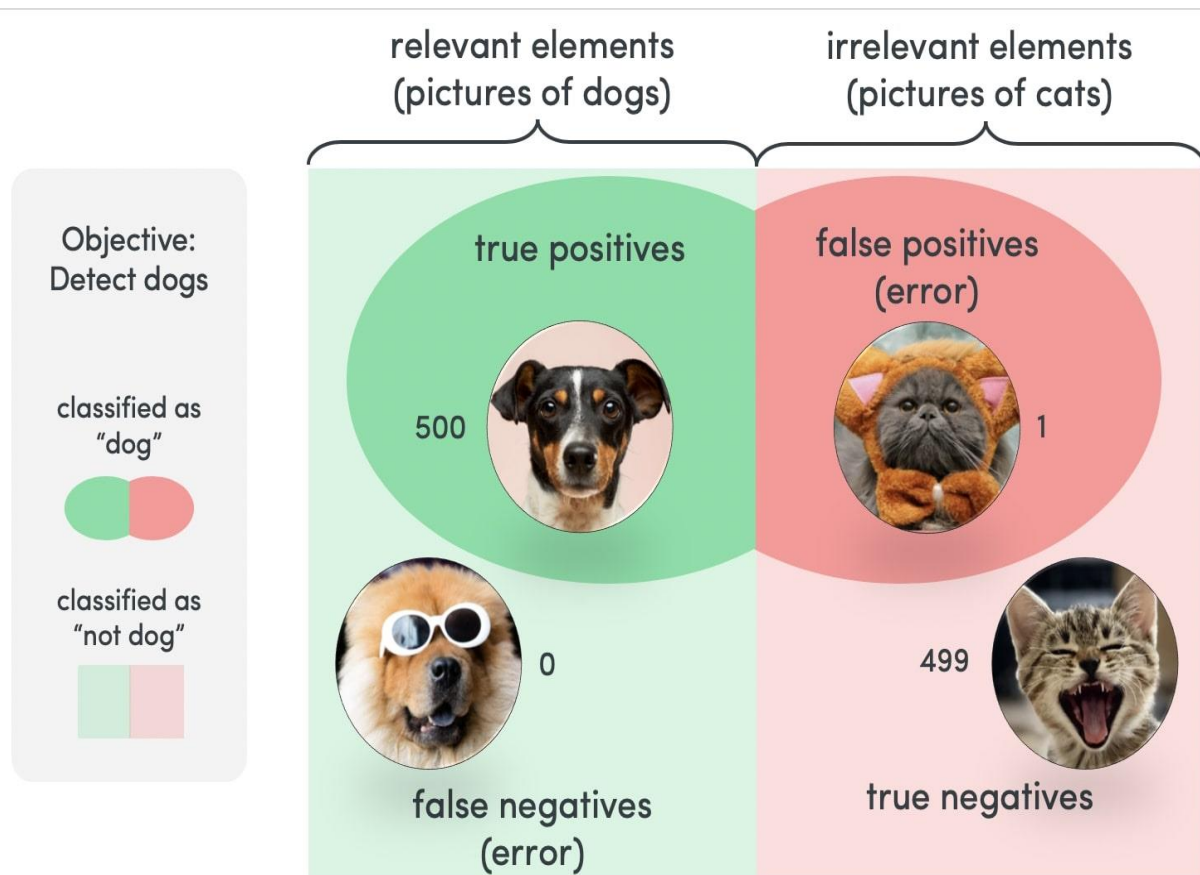
<https://yolov8.com/>

Performance Metrics

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Also known as **Recall**

Performance Metrics – simple example



Self-study on performance measures

<https://oslo.vision/en/blog/what-is-precision-and-what-is-recall-when-training-computer-vision-ai-models/>

<https://www.ultralytics.com/blog/accuracy-precision-recall>

<https://roboflow.com/blog/object-detection-metrics>

Next Lecture

Object Detection: Viola-Jones Detector